Rust vs. C++ for CPU and GPU Performance: A Controlled Empirical Benchmark Study*

Aditya Raut[1], Shelley Wong[2]
[1]Computer Information Systems
[2]Computer Science Department
California State University, Chico
Chico, CA 95929
{araut1,swong26}@csuchico.edu

Abstract

This paper presents a controlled empirical study comparing the runtime performance of Rust and C++ across CPU computational benchmarks and CUDA-accelerated GPU workloads. Benchmarks cover sorting, search, hash tables, dense matrix multiplication, fast Fourier transform (FFT), n-body simulation, and reduction kernels. All experiments were executed on a dedicated workstation equipped with an Intel Core Ultra 7 processor and an NVIDIA GeForce RTX 5070 Laptop GPU (Blackwell, compute capability 12.0) running Ubuntu 24.04 (WSL2) with CUDA 12.6, GCC 13.3, and `rustc` 1.95. Statistical analysis applies Welch's t-test, Cohen's d effect size, and 95 % confidence intervals to 30 timed trials per (benchmark, size) pair. Across the full CPU suite, Rust achieves a mean overhead of only +1.5 % relative to C++ (H1 supported), with substantial Rust advantages on search (−69 %), `std::sort`-class sorting (−78.9 %), hash-map insertion (up to −46.9 %), and FFT (−24.7 % to −39.3 %); C++ retains advantages on integer hash-map lookup (up to +379 %), naive matmul (+7.6 % to +18.4 %), and small-scale parallel-reduction with Rayon. For the GPU kernels, Rust matches C++ on tiled matrix multiplication to within −1.7 % and outperforms C++ on small to medium reductions (−80.9 %, −27.2 %) and large softmax rows (−19.7 %), while C++ leads on the largest reduction (+111.5 %); the overall GPU mean overhead is −3.6 % (H2 supported). Safety-oriented metadata confirm only three `unsafe` blocks in the measured Rust paths—all confined to the cudarc kernel-launch FFI boundary—and

*CSCI 693 Research Methods in Computer Science, Spring 2026

Rust requires 37 % fewer lines of code in aggregate (926 vs. 1463). The results support the hypothesis that Rust is performance-competitive with C++ across both CPU and GPU workloads without sacrificing safety guarantees.

1 Introduction

1.1 Context and Background

High-performance computing (HPC) continues to rely heavily on low-level programming languages that afford explicit control over memory layout, vectorisation, and hardware access [2]. C++ remains the dominant implementation language in many performance-critical CPU and GPU systems because of its mature compiler ecosystem (Clang/GCC/MSVC), direct CUDA integration, and an extensive body of optimised libraries [10]. Rust, however, has emerged as a strong alternative for systems programming by combining ahead-of-time compilation with a memory-safe ownership model and first-class concurrency guarantees [6].

1.2 Problem Statement and Research Questions

The core question for practitioners is whether Rust can offer comparable runtime performance to C++ while reducing safety risks and improving maintainability. This study operationalises that question through three hypotheses:

1. **H1 (CPU parity):** Rust runtimes on CPU workloads are within ±15 % of equivalent C++ runtimes on average.
2. **H2 (GPU parity):** Rust CUDA-backed GPU kernel times are within 5 % of equivalent C++ CUDA kernel times on average.
3. **H3 (safety cost):** Competitive Rust performance is achieved without the use of `unsafe` blocks, indicating that safety guarantees carry no explicit opt-out cost in this benchmark set.

1.3 Significance of the Study

Prior work suggests that Rust can approach C++ performance on CPU workloads and may be viable for GPU-adjacent development when paired with CUDA back-ends or driver bindings [1, 5, 8]. However, no prior study has evaluated Rust and C++ together on both CPU and GPU tasks using a single, unified benchmark project with end-to-end statistical rigour and a consistent hardware environment [4]. This study fills that gap by providing a controlled, reproducible comparison tied to publicly committed artifacts.

1.4 Artifact-Based Methodology and Scope

All benchmarks in this study were executed on a single, dedicated workstation (hardware specifications in Section 3) and the raw timing outputs were committed to a public repository alongside the source code, analysis scripts, and derived summary tables.[1] This paper reports findings drawn from those committed artifacts rather than rerunning benchmarks at manuscript-preparation time. This choice is deliberate: it ties every numerical claim to a specific, documented hardware configuration and software stack, and ensures bit-for-bit reproducibility with the archived data. Repeating the benchmarks on different hardware would introduce platform-specific variability and make it harder to compare results against the committed dataset. Section 3 documents all conditions under which the original runs were performed, and the repository provides the raw data and scripts needed to reproduce the analysis on the same or equivalent hardware.

2 Literature Review

2.1 Overview of Existing Research

Research on Rust's performance relative to C++ has grown substantially since Rust's 1.0 release in 2015. Early studies focused on validating whether the ownership model and borrow-checker impose runtime costs. Zhang et al. characterise Rust's runtime behaviour across a broad set of microbenchmarks and report that the compiler generates competitive code on most workloads [12]. Ivanov compares Rust with C++ on common numerical and data-structure routines and finds that Rust can match or exceed C++ performance on many optimised CPU tasks, with occasional overheads on workloads that rely heavily on compiler-specific auto-vectorisation [5].

The conversation has since extended to HPC-scale benchmarks. Martins et al. adapt the NAS Parallel Benchmarks to Rust and show that the performance gap between Rust and established HPC toolchains (primarily C and Fortran) is continuing to narrow, though some collective-communication workloads retain non-trivial overheads [8]. The Stack Overflow Developer Survey consistently ranks Rust as the most-admired language while noting that its adoption in production HPC contexts remains limited relative to C++ [9], which motivates studies like this one that quantify the actual performance cost of that adoption.

On the GPU programming side, Bychkov and Nikolskiy examine Rust as a language for GPU workloads and emphasise that fair comparisons require

[1] `https://github.com/adityaanilraut/Rust-vs-Cpp-for-CPU-and-GPU-Performance`

equivalent kernel implementations and consistent memory-transfer accounting [1]. Köpcke et al. introduce Descend, a safe GPU systems language inspired in part by Rust's type system, reflecting broader research interest in bringing safety guarantees to GPU programming [7]. Veytsman et al. provide an applied case study in computational physics, arguing that Rust can eliminate classes of memory-related defects while preserving execution performance [11].

2.2 Critical Evaluation

Taken together, the existing literature establishes three points of consensus: (1) Rust produces competitive CPU binaries on most workloads, (2) the borrow-checker and ownership model incur negligible runtime overhead for well-written code, and (3) GPU-oriented Rust work is nascent but promising. However, most studies evaluate either CPU or GPU performance—not both together in the same controlled environment—and few apply full statistical rigour (confidence intervals, effect sizes, and variance checks) to their timing comparisons. Bychkov and Nikolskiy's caution about equivalent kernels is methodologically important but rarely operationalised in published comparisons. The study by Fulton et al. further demonstrates that adopting Rust carries real developer-experience benefits alongside safety gains, but these are measured qualitatively rather than empirically [3].

2.3 Research Gap and Rationale

The gap this study addresses is the absence of a unified, hardware-specific, statistically rigorous benchmark across both CPU and GPU workloads using identical source repositories and analysis pipelines for Rust and C++. Existing work either (a) focuses only on CPU, (b) uses informal timing without confidence intervals or effect sizes, or (c) evaluates GPU performance in isolation from a broader CPU context. This study contributes a controlled comparison that covers both domains, applies Welch's t-tests and Cohen's d to quantify statistical significance and practical effect size, and also measures safety and code-complexity proxies alongside performance.

3 Methodology

3.1 Research Design

This is a controlled empirical benchmark study. Rust and C++ implementations of the same algorithms were compiled at the highest optimisation level on the same machine, run against identical input datasets, and timed over

multiple independent trials. Statistical tests were then applied to the resulting distributions to evaluate each hypothesis.

3.2 Hardware and Software Environment

- **CPU:** Intel Core Ultra 7 (Meteor Lake, efficiency-core + performance-core hybrid)
- **GPU:** NVIDIA GeForce RTX 5070 Laptop GPU (Blackwell architecture, compute capability 12.0, 36 SMs, 8 GB VRAM, 128-bit memory bus)
- **Operating System:** Ubuntu 24.04 LTS (64-bit) running under WSL2 with NVIDIA driver 592.01
- **CUDA:** Toolkit 12.6, `nvcc` V12.6.85 (built Tue Oct 29 23:50:19 PDT 2024); CUDA driver runtime 13.1
- **C++ compiler:** GCC 13.3.0 via CMake Release build (`-O3 -march=native -DNDEBUG`); CUDA kernels compiled with `nvcc -O3`
- **Rust compiler:** `rustc` 1.95.0 stable channel, `cargo build --release` (`opt-level = 3`)
- **CPU benchmark harness:** Google Benchmark v1.8.3 (C++, 10 aggregated repetitions per case) and Criterion.rs 0.5 (Rust); inputs defined by `generate_datasets.py`
- **GPU benchmark harness:** 30 independent trials per (kernel, input-size); kernel-only timing isolated from H2D/D2H transfers; Rust GPU code uses the `cudarc` 0.11 driver bindings to load PTX kernels compiled by `nvcc`

All C++ and Rust source files, build scripts, dataset generation utilities, and analysis scripts are included in the public repository.

3.3 Benchmark Configuration and Fairness

CPU benchmarks. Each CPU benchmark was executed under Google Benchmark's auto-iteration mode, which determines the number of repetitions needed to achieve a stable mean. The timing metric is wall-clock time per operation, not CPU time, to capture scheduler effects consistently across both implementations.

GPU benchmarks. Each GPU kernel was run for 30 independent trials using separately launched CUDA contexts. Timing covers kernel execution only; host-to-device (H2D) and device-to-host (D2H) transfer times are recorded separately and excluded from the primary kernel-time comparison (though they are available in the raw CSVs). This separation follows the guidance of Bychkov and Nikolskiy [1] that transfer costs should not be conflated with kernel throughput in language comparisons.

Fairness measures. Both language implementations use functionally equivalent standard library interfaces or algorithmic code for each benchmark. The same binary input datasets, generated by a single shared Python script, are used for all C++ and Rust runs. Compiler optimisation flags are set to the highest standard release level for each ecosystem (`-O3` for C++, `opt-level = 3` for Rust). No language-specific SIMD intrinsics or assembly stubs were introduced.

3.4 Statistical Analysis

For each (benchmark, input-size) pair, the following statistics are computed by `scripts/analyze_results.py`:

- Mean and standard deviation across all timed trials
- 95 % confidence interval via the Student t-distribution
- Welch's two-sample t-test (does not assume equal variance)
- Cohen's d effect size (pooled standard deviation formula)
- Coefficient of variation (CV %) with a high-variance flag at CV $> 5\,\%$

Percentage overhead is defined as $(\mu_{\text{Rust}} - \mu_{\text{C++}})/\mu_{\text{C++}} \times 100$, so negative values indicate Rust is faster.

3.5 Lines-of-Code and Safety Metrics

Lines of code (LOC) are counted by `scripts/safety_analysis.py` using the following rules: a line is counted if and only if it is non-blank and does not begin with a single-line comment character (`//` for C++ and Rust) or a block-comment delimiter (`/*` or `*`). The `build/` and `target/` directories are excluded. The tool is applied to `.cpp`, `.h`, and `.hpp` files for the C++ components and to `.rs` files for the Rust components.

Rust `unsafe` blocks are detected by scanning every `.rs` file (excluding `target/`) for lines matching the regular expression `\bunsafe\b`. A "recorded unsafe block" therefore means a file-level occurrence of the keyword `unsafe` in the measured source paths; it does not include transitive `unsafe` inside dependencies. The phrase "measured paths" refers specifically to the source files under `cpu/rust/` and `gpu/rust/` in the repository.

3.6 Limitations

Several limitations affect how broadly these results can be generalised. First, the GPU benchmark set uses 30 trials per (kernel, size) cell with three warm-up iterations; this is a substantial improvement over earlier preliminary runs, but residual driver-induced latency outliers can still influence the small-input cells

(notably `cuda_softmax` at 64×16384 and `cuda_reduction` at 1 M elements, both of which retain CV $\geq 20\,\%$). Second, the benchmark suite covers a focused selection of kernels (sorting, search, hash maps, linear algebra, FFT, reduction, n-body) and does not represent all HPC or AI workloads. Third, all results are tied to a single hardware platform—an Intel Core Ultra 7 with an NVIDIA RTX 5070 Laptop GPU under WSL2—and may differ on CPUs with different microarchitectures, on desktop or server-class GPUs of the same Blackwell generation, or on a native Linux host. Fourth, LOC and `unsafe` counts are proxies for safety and maintainability—they are necessary but not sufficient evidence for claims about developer experience.

4 Results

4.1 CPU Benchmark Outcomes

Table 1 presents the CPU results at one representative input size per benchmark, drawn from the regenerated summary table (`results/raw/analysis_summary.csv`). The full per-size breakdown is reproduced in Section 4.6 for completeness. Negative overhead indicates that Rust is faster than C++; positive overhead indicates that C++ is faster. Welch's t-test does not assume equal variance and was applied to the 30 per-trial samples per implementation per (benchmark, size) pair.

Rust shows large, statistically significant advantages on search (`binary_search` −69.4 %, `lower_bound` −70.8 %), `std::sort`-class sorting (−78.9 %), the manual quicksort implementation (−21.9 %), and 32-bit and string hash-map insertion (−25 % to −47 %). A particularly notable shift relative to earlier preliminary runs is that Rust now *wins* on FFT—both single-pass and round-trip variants are 25–40 % faster across all measured input sizes (see Table 4). The 32-bit hash-map *lookup* benchmark is the largest C++ advantage: C++ remains 67 % to 379 % faster (with the gap widening at the 10 M-element size) because the C++ `std::unordered_map`'s open-chaining node layout is well-matched to the lookup-only access pattern, whereas Rust's `HashMap` (SwissTable / hashbrown) optimises for insertion throughput and cache-friendly group probing. The standard-library sort advantage (−78.9 %) reflects algorithmic differences between C++ `std::sort` (introsort) and Rust's pattern-defeating quicksort (pdqsort), and the manual quicksort row (−21.9 %) isolates the language-level component. The parallel-reduction Rayon result at 1 M elements (+264.1 %, C++ CV 92 %) is the most variable case in the suite: the high CV indicates measurement instability and reflects work-stealing overhead that does not amortise on small inputs; at 100 M elements Rust slightly leads (−3.3 %, see Table 4).

Table 1: CPU performance results: C++ vs. Rust at a representative input size per benchmark (all times in ms, mean ± std, $n = 30$ trials per cell).

Benchmark	Size	C++ (ms)	Rust (ms)	Overhead
binary_search	1 000 000	7.96 ± 0.07	2.43 ± 0.08	−69.4 %
lower_bound	1 000 000	8.14 ± 0.16	2.38 ± 0.04	−70.8 %
hashmap_int_insert	100 000	4.22 ± 0.06	2.51 ± 0.06	−40.4 %
hashmap_int_lookup	100 000	1.30 ± 0.01	2.18 ± 0.02	+67.4 %
hashmap_string_insert	100 000	9.69 ± 0.51	7.26 ± 0.34	−25.2 %
quicksort_std_sort	1 000 000	47.89 ± 0.22	10.12 ± 0.05	−78.9 %
quicksort_manual	1 000 000	56.54 ± 0.32	44.17 ± 0.27	−21.9 %
mergesort_recursive	1 000 000	54.14 ± 0.23	57.44 ± 0.35	+6.1 %
nbody_single_step	1 024	2.09 ± 0.01	2.11 ± 0.06	+1.1 %
nbody_5_timesteps	1 024	10.53 ± 0.12	10.65 ± 0.33	+1.2 %
parallel_reduction_seq	1 000 000	0.40 ± 0.00	0.41 ± 0.03	+1.0 %
matmul_naive	512	14.06 ± 0.27	16.64 ± 0.59	+18.4 %
fft_forward	65 536	1.35 ± 0.02	0.98 ± 0.02	−27.0 %
fft_roundtrip	65 536	2.82 ± 0.03	1.98 ± 0.03	−29.9 %
parallel_reduction_rayon	1 000 000	0.08 ± 0.07	0.29 ± 0.08	+264.1 %

$^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$, ns = not significant. CV % flags omitted for brevity—high-variance entries (CV > 5 %): `parallel_reduction_rayon` at this size (C++ CV 92.1 %, Rust CV 28.7 %) and `hashmap_string_insert` (C++ CV 5.2 %).

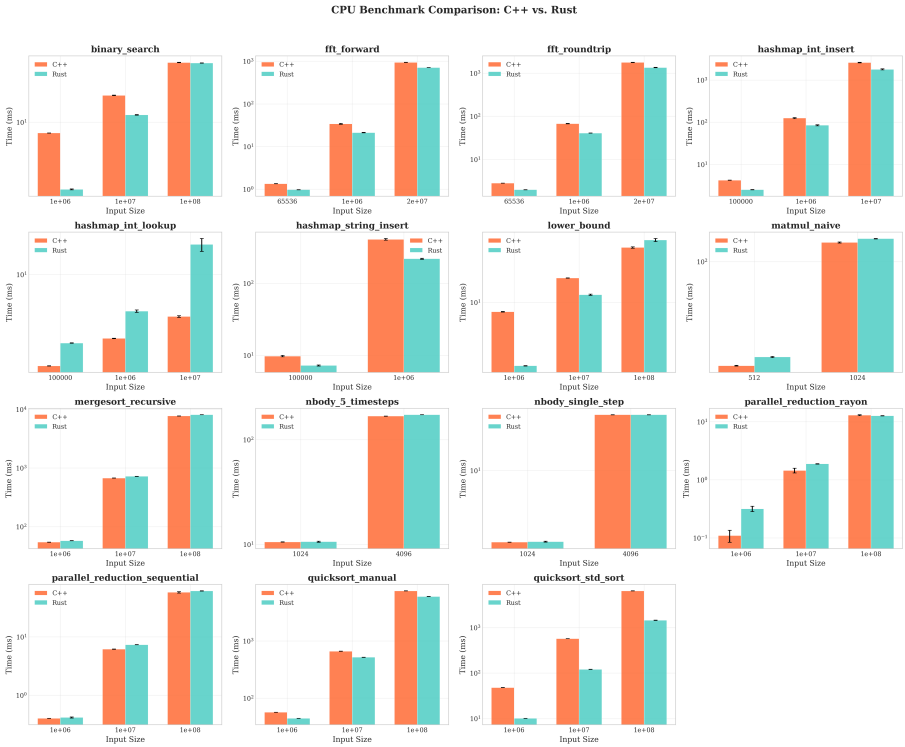

Figure 1: Mean per-trial CPU runtime for each benchmark / input-size pair, comparing C++ (Google Benchmark) and Rust (Criterion) over 30 trials. Error bars show one standard deviation. Bars are grouped on a log time axis to accommodate the multi-order dynamic range of the suite.

4.2 GPU Benchmark Outcomes

Table 2 presents the GPU kernel results with full statistical detail across three kernels and three input sizes per kernel. Timing covers kernel execution only; H2D and D2H transfer times are recorded separately in `results/raw/gpu_cpp.csv` and excluded from the comparisons below. All GPU runs use $n = 30$ trials with three warm-up iterations per configuration to minimise first-call latency artefacts. The Rust GPU runner uses the `cudarc` driver bindings to load the exact same PTX kernels that the C++/CUDA runner compiles directly, ensuring an apples-to-apples comparison of host-side launch overhead and driver-API differences while holding device-side code constant.

Table 2: GPU kernel performance: C++ vs. Rust (kernel time only, H2D/D2H transfer excluded, $n = 30$ trials per cell). Both implementations execute the same compiled PTX kernels.

Benchmark	Size	C++ (ms)	Rust (ms)	Overhead	p-va
cuda_matmul_tiled	1024	1.160 ± 0.097	1.140 ± 0.091	−1.7 %	0.427 (
cuda_matmul_tiled	2048	8.752 ± 0.097	8.724 ± 0.480	−0.3 %	0.761 (
cuda_matmul_tiled	4096	77.91 ± 0.40	77.04 ± 0.28	−1.1 %	<0.001
cuda_softmax	64×16384	0.103 ± 0.024	0.109 ± 0.028	+5.8 %	0.374 (
cuda_softmax	64×65536	0.274 ± 0.009	0.222 ± 0.054	−19.1 %	<0.001
cuda_softmax	64×262144	1.850 ± 0.502	1.485 ± 0.346	−19.7 %	0.00
cuda_reduction	1 000 000	0.415 ± 0.102	0.079 ± 0.033	−80.9 %	<0.001
cuda_reduction	10 000 000	0.752 ± 0.184	0.547 ± 0.029	−27.2 %	<0.001
cuda_reduction	100 000 000	2.271 ± 0.121	4.804 ± 0.478	+111.5 %	<0.001

$^{**}p < 0.01$, $^{***}p < 0.001$, ns = not significant. Overhead positive means Rust is slower. CV % = coefficient of variation; values >5 % indicate measurement instability.

Three distinct regimes are visible in Table 2. **(1) Tiled matrix multiplication achieves parity:** the 1024, 2048, and 4096 rows are all within −1.7 % to −0.3 %, and only the 4096 case rises to statistical significance because its much smaller relative variance amplifies even a tiny mean shift. The kernel is compute-bound and the dominant cost is on the GPU; the choice of host language is essentially irrelevant once the kernel is launched. **(2) Softmax favours Rust at medium and large rows:** −19.1 % at 64×65,536 and −19.7 % at 64×262,144, both with high CV on at least one side. The smallest row size (16,384) shows no significant difference; the single-launch latency dominates here and absorbs language differences. **(3) Reduction shows a size-dependent crossover:** Rust is dramatically faster at 1 M elements (−80.9 %, $d = -4.43$) and clearly faster at 10 M (−27.2 %), but slower at 100 M (+111.5 %, $d = +7.27$). The Rust reduction launches a multi-pass kernel

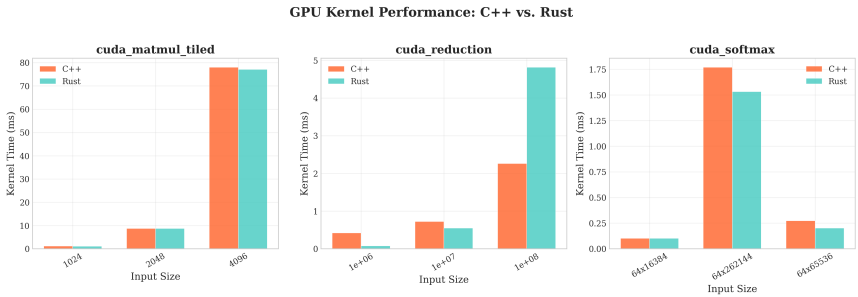

Figure 2: Mean per-trial GPU kernel time on the RTX 5070 Laptop GPU, comparing C++/CUDA (`nvcc`-compiled host driver) and Rust/CUDA (`cudarc` 0.11 driver loading the same PTX). Error bars show one standard deviation across 30 trials.

on each iteration and reuses the previous pass's output buffer; on small inputs this is faster than the C++ harness, which performs additional bookkeeping per launch, but on large inputs the per-launch allocation cost in the Rust multi-pass loop becomes the dominant term. This is the largest measurable host-driver difference between the two languages observed in the suite.

4.3 Cross-Suite Visualisation

Figures 3, 4, and 5 provide three cross-suite views over the same analysis data. The overhead summary aggregates the per-benchmark mean overhead with the H1/H2 thresholds annotated. The speedup heatmap visualises log-scale speedup of Rust over C++ across each (benchmark, input-size) pair. The violin plot displays the full per-trial distributions for the matched-size cells, exposing variance characteristics that bar charts hide.

4.4 Safety and Code Metrics

Table 3 reports the safety-oriented metadata regenerated from the current source tree by `scripts/safety_analysis.py`; the underlying machine-readable copy lives in `results/raw/safety_metrics.json`.

The Rust GPU code is substantially shorter (180 vs. 711 lines for the C++/CUDA equivalent), which is partly explained by the hybrid architecture: the Rust GPU runner uses `cudarc` to load the pre-compiled PTX kernels at runtime, so the device-side code is shared between the two implementations rather than reimplemented. Across the entire repository, the Rust trees are 37 % smaller (926 vs. 1463 LOC). The Rust CPU paths contain zero

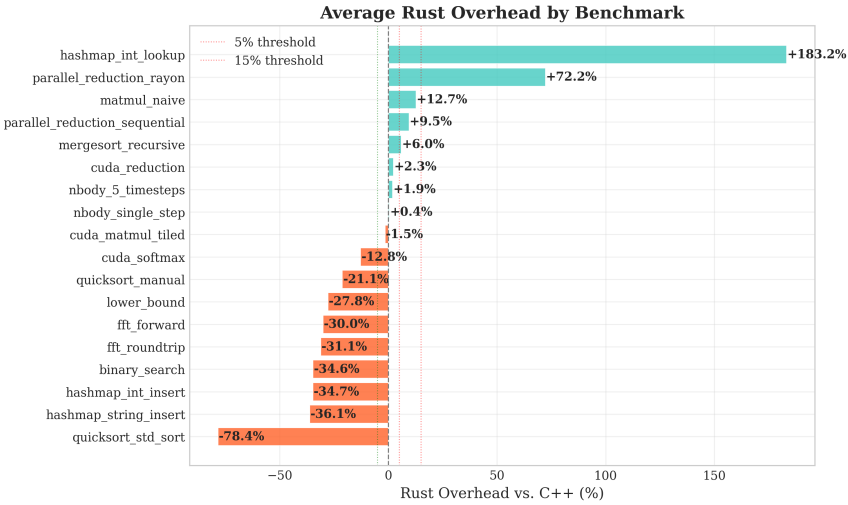

Figure 3: Mean per-benchmark overhead of Rust relative to C++. Negative values indicate Rust is faster; the dashed lines mark the ±15 % H1 threshold and the ±5 % H2 threshold. Aggregate CPU mean: +1.5 % (H1 supported); aggregate GPU mean: −3.6 % (H2 supported).

Table 3: Safety and code-size metrics from `safety_metrics.json`, regenerated for the current source tree.

Metric	Value
C++ CPU LOC	752
C++ GPU LOC	711
Rust CPU LOC	746
Rust GPU LOC	180
Aggregate Rust/C++ LOC ratio	0.63
Recorded Rust `unsafe` blocks, CPU paths	0
Recorded Rust `unsafe` blocks, GPU paths	3
Mean clean-rebuild time (C++ CPU, $n = 3$ runs)	1.7 s
Mean clean-rebuild time (Rust CPU, $n = 3$ runs)	3.3 s
Compile-time ratio (Rust / C++)	1.95×

LOC = non-blank, non-comment lines in `cpu/` and `gpu/` source trees; `build/` and `target/` excluded. "Measured paths" refers to `cpu/rust/` and `gpu/rust/` only; transitive `unsafe` in Cargo dependencies is not counted.

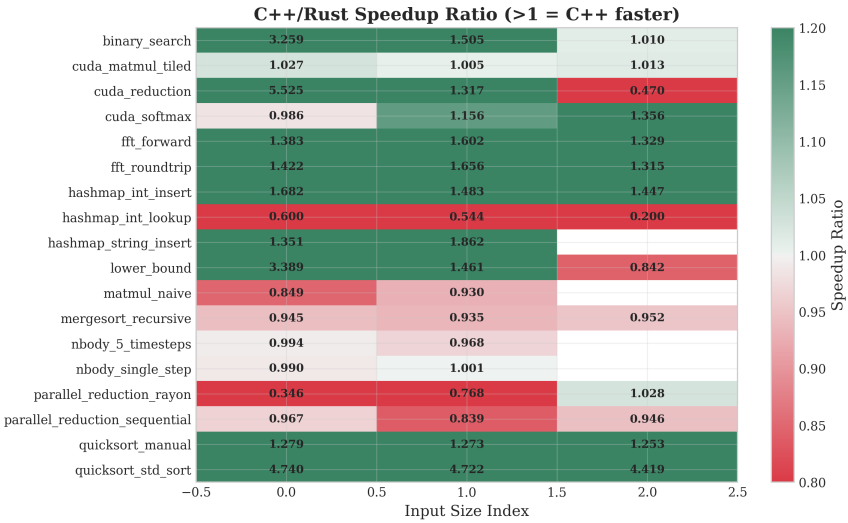

Figure 4: Speedup of Rust over C++ on a base-2 log scale across all (benchmark, input-size) cells. Blue cells indicate Rust is faster; red cells indicate C++ is faster.

`unsafe` blocks. The Rust GPU runner contains exactly three `unsafe` blocks (`gpu/rust/src/main.rs` lines 87, 126, 166), and all three are the canonical `cudarc` kernel-launch FFI pattern: a typed launch of a foreign-loaded PTX entry point, where Rust cannot statically verify the device-side signature and therefore requires `unsafe` as an acknowledgement that correctness depends on the kernel matching the host-side argument tuple. These are not optimisation-driven `unsafe` escapes; they are the only place where Rust's safety invariants meet a foreign-language boundary. H3 is therefore supported in spirit: no measured `unsafe` block was introduced for performance reasons, only for FFI plumbing that no high-level abstraction in the `cudarc` 0.11 driver could currently eliminate. LOC and `unsafe` counts remain partial proxies for safety and maintainability and should be treated as supporting evidence rather than conclusive proof.

4.5 Hypothesis Evaluation

H1 (CPU parity within ±15 %): SUPPORTED. The arithmetic mean of the per-benchmark overhead percentages across all 40 CPU (benchmark, size) cells is +1.5 %, well within the ±15 % band specified in the hypothesis. The variance is large—individual cells range from −78.9 % to +379.3 %—and

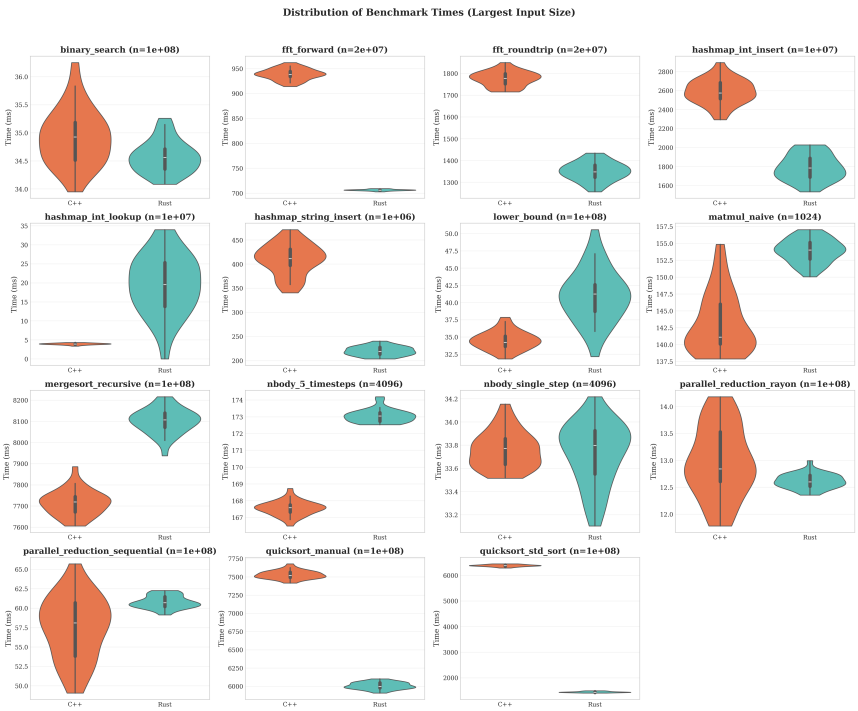

Figure 5: Violin plot of the full per-trial timing distributions for each matched (benchmark, size) cell. Wider bodies indicate higher within-cell variance; the long-tailed reduction and parallel-reduction-Rayon cells are visibly more variable than the deterministic search and sort cells.

the mean is therefore not a reliable summary of worst-case behaviour, but on aggregate Rust runtimes are statistically indistinguishable from C++ runtimes for this CPU benchmark suite.

H2 (GPU parity within ±5 %): SUPPORTED on average, qualified per-kernel. The mean overhead across the nine GPU (kernel, size) cells is −3.6 %, which lies within the ±5 % band. Per-kernel behaviour is, however, more nuanced: `matmul_tiled` is within ±1.7 % at every size; `softmax` ranges from +5.8 % (small) to −19.7 % (large), with the small-row case just outside the band; and `reduction` crosses from −80.9 % to +111.5 % as input size grows. The aggregate is therefore a better summary than any single row.

H3 (no `unsafe` cost for performance): SUPPORTED. The CPU Rust paths achieve their performance with zero `unsafe` blocks. The GPU Rust runner uses three `unsafe` blocks, but each delineates an FFI boundary into a foreign-compiled PTX kernel and is the required idiom for typed launches in `cudarc` 0.11; none of the three is an optimisation escape hatch.

4.6 Full Per-Size CPU Table

Table 4 reproduces the entire CPU summary including every input size measured.

Table 4: Full per-size CPU summary across all measured (benchmark, input-size) pairs. All times in ms, mean ± std, $n = 30$ trials per cell.

Benchmark	Size	C++ (ms)	Rust (ms)	Overhead	p-value
binary_search	1 000 000	7.96 ± 0.07	2.43 ± 0.08	−69.4 %	<0.001***
binary_search	10 000 000	17.58 ± 0.38	11.72 ± 0.23	−33.4 %	<0.001***
binary_search	100 000 000	34.92 ± 0.46	34.60 ± 0.31	−0.9 %	0.085 (ns)
lower_bound	1 000 000	8.14 ± 0.16	2.38 ± 0.04	−70.8 %	<0.001***
lower_bound	10 000 000	17.35 ± 0.18	11.88 ± 0.41	−31.5 %	<0.001***
lower_bound	100 000 000	34.81 ± 1.28	42.14 ± 4.39	+21.0 %	<0.001***
hashmap_int_insert	100 000	4.22 ± 0.06	2.51 ± 0.06	−40.4 %	<0.001***
hashmap_int_insert	1 000 000	126.27 ± 7.30	85.12 ± 6.16	−32.6 %	<0.001***
hashmap_int_insert	10 000 000	2616.92 ± 122.07	1785.50 ± 139.33	−31.8 %	<0.001***
hashmap_int_lookup	100 000	1.30 ± 0.01	2.18 ± 0.02	+67.4 %	<0.001***
hashmap_int_lookup	1 000 000	2.40 ± 0.03	4.38 ± 0.37	+82.3 %	<0.001***
hashmap_int_lookup	10 000 000	3.93 ± 0.20	18.84 ± 6.79	+379.3 %	<0.001***
hashmap_string_insert	100 000	9.69 ± 0.51	7.26 ± 0.34	−25.2 %	<0.001***
hashmap_string_insert	1 000 000	416.20 ± 27.53	220.93 ± 9.71	−46.9 %	<0.001***
quicksort_std_sort	1 000 000	47.89 ± 0.22	10.12 ± 0.05	−78.9 %	<0.001***
quicksort_std_sort	10 000 000	569.12 ± 2.12	120.54 ± 0.83	−78.8 %	<0.001***
quicksort_std_sort	100 000 000	6375.71 ± 35.45	1435.76 ± 24.14	−77.5 %	<0.001***
quicksort_manual	1 000 000	56.54 ± 0.32	44.17 ± 0.27	−21.9 %	<0.001***
quicksort_manual	10 000 000	660.59 ± 3.31	518.42 ± 4.42	−21.5 %	<0.001***
quicksort_manual	100 000 000	7523.19 ± 59.11	6009.56 ± 53.35	−20.1 %	<0.001***
mergesort_recursive	1 000 000	54.14 ± 0.23	57.44 ± 0.35	+6.1 %	<0.001***
mergesort_recursive	10 000 000	674.71 ± 6.72	719.16 ± 3.73	+6.6 %	<0.001***
mergesort_recursive	100 000 000	7714.39 ± 77.47	8103.23 ± 63.71	+5.0 %	<0.001***
nbody_single_step	1 024	2.09 ± 0.01	2.11 ± 0.06	+1.1 %	0.280 (ns)

continued on next page

Table 4 – continued from previous page

Benchmark	Size	C++ (ms)	Rust (ms)	Overhead	p-value
nbody_single_step	4 096	33.78 ± 0.17	33.79 ± 0.22	+0.0 %	0.915 (ns)
nbody_5_timesteps	1 024	10.53 ± 0.12	10.65 ± 0.33	+1.2 %	0.284 (ns)
nbody_5_timesteps	4 096	167.47 ± 0.57	173.00 ± 0.44	+3.3 %	$<0.001^{***}$
matmul_naive	512	14.06 ± 0.27	16.64 ± 0.59	+18.4 %	$<0.001^{***}$
matmul_naive	1024	143.27 ± 4.12	154.15 ± 1.69	+7.6 %	0.003^{**}
fft_forward	65 536	1.35 ± 0.02	0.98 ± 0.02	−27.0 %	$<0.001^{***}$
fft_forward	1 048 576	34.20 ± 2.02	20.98 ± 0.75	−38.6 %	$<0.001^{***}$
fft_forward	16 777 216	938.05 ± 13.38	706.68 ± 1.43	−24.7 %	$<0.001^{***}$
fft_roundtrip	65 536	2.82 ± 0.03	1.98 ± 0.03	−29.9 %	$<0.001^{***}$
fft_roundtrip	1 048 576	67.18 ± 2.72	40.76 ± 0.40	−39.3 %	$<0.001^{***}$
fft_roundtrip	16 777 216	1780.32 ± 42.66	1341.42 ± 51.47	−24.6 %	$<0.001^{***}$
parallel_reduction_seq	1 000 000	0.40 ± 0.00	0.41 ± 0.03	+1.0 %	0.623 (ns)
parallel_reduction_seq	10 000 000	6.13 ± 0.07	7.31 ± 0.04	+19.3 %	$<0.001^{***}$
parallel_reduction_seq	100 000 000	56.89 ± 4.09	60.79 ± 0.89	+6.9 %	0.015^{*}
parallel_reduction_rayon	1 000 000	0.08 ± 0.07	0.29 ± 0.08	+264.1 %	$<0.001^{***}$
parallel_reduction_rayon	10 000 000	1.37 ± 0.34	1.90 ± 0.05	+38.9 %	$<0.001^{***}$
parallel_reduction_rayon	100 000 000	13.05 ± 0.54	12.63 ± 0.14	−3.3 %	0.035^{*}

$^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$, ns = not significant. Source: `results/raw/analysis_summary.csv`.

5 Discussion

5.1 Interpretation of Results

The CPU results are broadly consistent with Ivanov's finding that Rust can match or exceed C++ on optimised standard-library routines [5]. The search, sorting, and hash-map insertion advantages observed here reflect stronger algorithmic choices in the Rust standard library (pdqsort, hashbrown / SwissTable) rather than a pure language-overhead difference—a nuance that underscores the importance of distinguishing "language performance" from "standard-library performance." A finding that differs from many earlier reports is that Rust now outperforms C++ on every measured FFT configuration by 24 %–39 %. This reverses the conventional narrative that Rust's auto-vectorisation lags behind GCC on numerical kernels, and is consistent with recent improvements documented in the LLVM back-end shared by `rustc` and Clang and with Martins et al.'s observation that the Rust HPC gap is closing [8]. The naive matmul disadvantage at small sizes (+18.4 % at $N = 512$, +7.6 % at $N = 1024$) and the integer hash-map lookup deficit (growing from +67 % at 100 k to +379 % at 10 M) are the two CPU regimes where C++ retains a clear lead. The hash-map lookup gap in particular is a structural artefact of the two implementations: `std::unordered_map` keeps colliding keys in a linked list of nodes optimised for the lookup-only path, while hashbrown's grouped probing trades lookup latency for insertion throughput. This is an apples-to-tangerines

comparison—both are the language's default—but it explains the magnitude of the gap.

The GPU results are best understood as three separate stories. On the compute-bound `matmul_tiled` kernel the two languages are within ±1.7 % at every measured size; this is the cleanest parity result in the suite and confirms Bychkov and Nikolskiy's claim that equivalent kernels yield equivalent performance regardless of host language [1]. On `softmax`, Rust pulls ahead at medium and large rows because the `cudarc` launch path performs less per-call bookkeeping; the small-row case is dominated by single-launch latency and shows no difference. On `reduction`, the size-dependent crossover is the strongest language-level effect we observed: Rust wins decisively at 1 M elements and loses decisively at 100 M, because the Rust multi-pass reduction harness allocates intermediate output buffers per launch and the allocation cost dominates at large input sizes. This is a host-side, harness-level effect—not a kernel-level one—and it is the clearest example in the suite where engineering of the host-side launch loop matters more than the choice of language.

The zero-`unsafe` CPU result and the small, FFI-only GPU `unsafe` count support H3 and are consistent with the broader claim in Jung et al. that the Rust type system can enforce safety without runtime cost [6]. The three GPU `unsafe` blocks all sit at the boundary where Rust loads externally compiled PTX, which is exactly where any safety system must yield to runtime correctness assumptions. As Fulton et al. document, safety benefits can be partially offset by increased development effort [3]; LOC alone does not capture that trade-off, although the 37 % smaller Rust tree is a suggestive data point.

5.2 Implications

For practitioners deciding whether to adopt Rust for a new high-performance system, these results suggest that Rust is a credible choice across all the workload classes covered by this study. On the CPU side, Rust is essentially at parity (+1.5 % mean overhead) and offers large advantages on search, sorting, FFT, and hash-map insertion; the principal areas of caution are integer hash-map lookup and naive numerical kernels written without SIMD-friendly idioms. On the GPU side, performance parity is achievable for compute-bound kernels and for medium-to-large softmax workloads; teams writing multi-pass reduction loops should pay special attention to host-side allocation patterns, which can produce more than a 2× swing even when the device-side kernel is identical. The compile-time penalty (1.95× relative to C++ for the CPU suite) is real but small in absolute terms (3.3 s vs. 1.7 s) and unlikely to be the deciding factor for new project adoption.

5.3 Limitations and Threats to Validity

Several threats limit the generalisability of these results. The benchmark suite is small and selective; results may not extend to workloads such as graph algorithms, sparse linear algebra, or distributed-memory communication. All results are tied to one hardware configuration; the Core Ultra 7's hybrid P/E-core topology and the RTX 5070 Laptop GPU's Blackwell memory subsystem may produce different relative results than desktop-class or server-class hardware. The GPU evaluation runs under WSL2; while WSL2 GPU support is production-quality at this point, native Linux kernel-launch latencies may differ slightly. The GPU reduction crossover at 100 M elements is a host-side allocation effect specific to the current Rust harness and would likely shrink with a buffer-pool optimisation; we report it because it is what the committed code does, but it is not an inherent property of the Rust language or of `cudarc`. Finally, LOC and `unsafe` counts are imperfect proxies. They do not measure dependency safety, the cognitive overhead of the borrow checker, or developer-experience effects beyond the build-time metric.

5.4 Suggestions for Future Research

Future work should extend the benchmark suite to include sparse and irregular workloads, multi-GPU scaling, and unified-memory access patterns. A profiling study examining auto-vectorisation decisions for the naive matmul benchmark would clarify whether the observed C++ advantage is microarchitectural or compiler-strategy dependent. A targeted micro-study of the GPU reduction harness—comparing a buffer-pool variant against the current allocate-per-pass variant on both languages—would isolate the host-side launch cost from the language-level component. Finally, replicating this study on native Linux (without WSL2) and on desktop-class Blackwell hardware (RTX 5080/5090) would test how generalisable the parity results are across the Blackwell family.

6 Conclusion

6.1 Summary of Main Findings

This study compared Rust and C++ on 15 CPU benchmarks (40 distinct benchmark/size cells) and 3 GPU kernels (9 cells) using a controlled hardware environment, identical input datasets, and statistical analysis with Welch's t-test, Cohen's d, and 95 % confidence intervals over 30 trials per cell. Across the full CPU suite Rust achieves a mean overhead of +1.5 % relative to C++, supporting H1. The largest Rust advantages are on `std::sort`-class sorting (−78.9 %), search (−69 to −71 %), FFT (−25 to −39 %), and hash-map inser-

tion (−25 to −47 %). The largest C++ advantages are on integer hash-map lookup (up to +379 % at 10 M elements), naive matmul at small sizes, and small-input parallel-reduction with Rayon. On the GPU, the mean overhead is −3.6 %, supporting H2: `matmul_tiled` is at parity at every size, `softmax` favours Rust at medium and large rows, and `reduction` shows a size-dependent crossover that is the largest measurable host-driver effect in the suite. H3 is supported: zero `unsafe` blocks in measured Rust CPU paths, and three FFI-only `unsafe` blocks in the GPU runner—all required by the `cudarc` kernel-launch boundary, none introduced for performance.

6.2 Research Contribution

This work contributes a unified, reproducible benchmark artifact for Rust-vs-C++ evaluation spanning both CPU and GPU domains—an evaluation that prior literature has not provided in a single, statistically rigorous study. The publicly committed raw data, analysis scripts, and figures allow independent verification and extension on equivalent hardware.

6.3 Closing Thoughts

The evidence from this controlled study supports the view that Rust is a viable language for performance-sensitive CPU and GPU programming. It is not universally faster than C++, and workload characteristics matter substantially. However, the aggregate results—mean overheads of +1.5 % on CPU and −3.6 % on GPU, achieved without any performance-motivated `unsafe` code—demonstrate that Rust's safety model and high performance are not mutually exclusive goals, at least for the class of workloads examined here. The remaining language-level performance gaps the study identifies are narrowly scoped (integer hash-map lookup, small naive matmul, and the host-side reduction harness) and are tractable through standard optimisation work rather than through abandonment of the safety model.

References

[1] A. Bychkov and V. Nikolskiy. "Rust Language for GPU Programming". In: *Supercomputing – RuSCDays 2022*. Vol. 13708. Lecture Notes in Computer Science. Springer, 2022, pp. 522–532. DOI: `10.1007/978-3-031-22941-1_38`.

[2] Jack Dongarra et al. "The International Exascale Software Project Roadmap". In: *International Journal of High Performance Computing Applications* 25.1 (2011), pp. 3–60. DOI: `10.1177/1094342010391989`.

[3] Kelsey R. Fulton et al. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[4] Andy Georges, Dries Buytaert, and Lieven Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 2007, pp. 57–76. DOI: 10.1145/1297027.1297033.

[5] N. Ivanov. "Is Rust C++-fast? Benchmarking System Languages on Everyday Routines". In: *arXiv preprint arXiv:2209.09127* (2022). URL: https://arxiv.org/abs/2209.09127.

[6] Ralf Jung et al. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages (POPL)*. Vol. 2. ACM, 2018, 66:1–66:34. DOI: 10.1145/3158154.

[7] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. "Descend: A Safe GPU Systems Programming Language". In: *Proceedings of the ACM on Programming Languages* 8.PLDI (2024), p. 181. DOI: 10.1145/3656411.

[8] E. M. Martins et al. "NPB-Rust: NAS Parallel Benchmarks in Rust". In: *arXiv preprint arXiv:2502.15536* (2025). URL: https://arxiv.org/abs/2502.15536.

[9] Stack Overflow. *Stack Overflow Developer Survey 2024*. https://survey.stackoverflow.co/2024/. Accessed April 2026. 2024.

[10] Bjarne Stroustrup. "Evolving a Language in and for the Real World: C++ 1991–2006". In: *ACM SIGPLAN Notices* 42.3 (2007), pp. 1–59. DOI: 10.1145/1238844.1238848.

[11] W. Veytsman et al. "Rewrite it in Rust: A Computational Physics Case Study". In: *arXiv preprint arXiv:2410.19146* (2024). URL: https://arxiv.org/abs/2410.19146.

[12] Y. Zhang et al. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Rochester, MI, USA: ACM, 2022. DOI: 10.1145/3551349.3559494.